

Evaluación perezosa y gestión de memoria en Haskell

Práctica Profesional Supervisada (PPS)

Agosto de 2026

Propósito. Este marco teórico vincula la semántica no estricta de Haskell con su implementación en GHC, la representación de la evaluación perezosa y la gestión de memoria del Runtime System, con el objetivo de proporcionar criterios para interpretar los experimentos de la PPS.

Alcance y organización

El esquema original de ocho unidades se reorganizó en seis capítulos para evitar repetir conceptos que pertenecen al mismo nivel de implementación. La representación operacional de la evaluación perezosa se distribuye entre las Unidades 2, 3 y 4, mientras que los costos y las optimizaciones se integran en la Unidad 5. La Unidad 6 presenta evaluación híbrida como trabajo relacionado y conecta el marco con los tres experimentos ya definidos.

El foco se mantiene en evaluación, representación y memoria. Scheduling general del RTS, STM, FFI, señales y excepciones asincrónicas no se desarrollan porque no forman parte del protocolo experimental; la concurrencia aparece únicamente donde modifica el ciclo de vida de los thunks, en particular al estudiar black holes. Esta delimitación evita extender el marco hacia mecanismos del RTS que no aportan evidencia directa a las preguntas de la PPS.

Alcance de la unidad. Este capítulo presenta el vocabulario conceptual necesario para estudiar, en las unidades siguientes, la evaluación por necesidad, la representación mediante closures y thunks, y sus efectos sobre la asignación y retención de memoria en GHC.

1. Lenguajes funcionales y Haskell

La programación funcional concibe la ejecución de un programa principalmente como la evaluación y composición de expresiones. En este paradigma, las funciones no son únicamente bloques de código que se invocan para modificar un estado externo: constituyen valores que pueden construirse, combinarse, recibir argumentos y producir nuevos resultados. Esta perspectiva tiene sus raíces formales en el cálculo lambda y en los modelos de reducción de expresiones desarrollados para implementar lenguajes funcionales (Peyton Jones, 1987). Los lenguajes modernos extendieron esa base mediante sistemas de tipos polimórficos y tipos de datos algebraicos. Maessen (2002, pp. 18-19) utiliza precisamente estas propiedades -funciones de orden superior, tipado polimórfico y tipos algebraicos- para caracterizar a esta familia de lenguajes.

Haskell ocupa un lugar particular dentro de esta tradición porque combina programación funcional pura, tipado estático y semántica no estricta. Estas decisiones favorecen una descripción declarativa de los problemas y permiten razonar sobre los programas mediante igualdades, pero también trasladan responsabilidades importantes al compilador y al sistema de ejecución. En particular, una implementación debe decidir cuándo evaluar cada expresión, cómo representar los cómputos pendientes y cómo administrar los objetos que se crean durante ese proceso. Por esta razón, los conceptos introductorios de la presente unidad no son independientes del problema de memoria estudiado en la PPS: forman la base semántica de las estructuras que más adelante aparecerán en el heap de GHC (Marlow y Peyton Jones, 2012).

1.1. Funciones como valores y abstracción funcional

En un lenguaje funcional, una función puede tratarse como cualquier otro valor del programa. Puede asignarse a un nombre, almacenarse dentro de una estructura, enviarse como argumento o devolverse como resultado. Cuando una función recibe o produce otras funciones se la denomina función de orden superior. Esta capacidad permite separar el patrón general de un cálculo de la operación concreta que se desea realizar, reduciendo duplicación y facilitando la reutilización.

La función `map` es un ejemplo representativo: recibe una transformación y la aplica a cada elemento de una lista. Su tipo expresa que la operación puede transformar valores de un tipo `a` en valores de otro tipo `b`, sin depender de cuáles sean esos tipos concretos:

```
map :: (a -> b) -> [a] -> [b]

duplicar :: Int -> Int
duplicar x = 2 * x

resultado = map duplicar [1, 2, 3]    -- [2, 4, 6]
```

En este caso, `duplicar` es pasado como dato a `map`. La misma estructura de recorrido puede reutilizarse con cualquier función compatible. La abstracción no surge solo de ocultar instrucciones, sino de parametrizar el comportamiento del programa con otras funciones. Según el informe de Haskell, las expresiones lambda permiten además construir funciones anónimas, por ejemplo, `map (\x -> 2 * x) [1, 2, 3]` (Marlow, 2010, sec. 3).

1.2. Currificación y aplicación parcial

Las funciones de Haskell están currificadas: una función que aparenta recibir varios argumentos puede interpretarse como una sucesión de funciones de un solo argumento. La flecha de los tipos es asociativa hacia la derecha, por lo que `Int -> Int -> Int` equivale a `Int -> (Int -> Int)`. Así, al aplicar `suma` al primer argumento no se obtiene todavía un número, sino una nueva función que espera el argumento restante.

```
suma :: Int -> Int -> Int
suma x y = x + y

sumarDiez :: Int -> Int
sumarDiez = suma 10

resultado = sumarDiez 7    -- 17
```

La currificación describe la forma en que se interpreta una función de múltiples parámetros; la aplicación parcial es el uso de esa propiedad para fijar solo una parte de sus argumentos. Aunque ambos conceptos están estrechamente relacionados, no son sinónimos. Esta distinción será relevante al estudiar la implementación: una aplicación parcial debe conservar tanto la función original como los argumentos ya recibidos, información que GHC representa en tiempo de ejecución mediante una closure especializada. Las unidades posteriores retomarán este vínculo al presentar las aplicaciones parciales o PAP y el modelo `eval/apply`.

1.3. Pureza, inmutabilidad y transparencia referencial

Una función pura produce el mismo resultado para los mismos argumentos y no modifica un estado observable ajeno a su resultado. De esta propiedad se deriva la transparencia referencial: una expresión

puede sustituirse por su valor sin cambiar el significado del programa. Por ejemplo, si cuadrado 4 siempre representa 16, reemplazar una aparición por la otra no altera el comportamiento. Esta posibilidad de sustitución constituye la base del razonamiento ecuacional y habilita transformaciones tanto manuales como automáticas (Maessen, 2002, pp. 21-22).

La pureza se relaciona con la inmutabilidad. En el nivel del lenguaje, un nombre designa un valor y no una celda cuyo contenido cambia arbitrariamente. Crear una versión modificada de una estructura produce conceptualmente un nuevo valor. Esto no significa que una implementación deba copiar toda la estructura: al no existir una modificación observable, es posible compartir componentes inalterados y efectuar ciertas actualizaciones internas de manera controlada. Más adelante, esta observación permitirá comprender por qué el runtime puede reemplazar un thunk evaluado por su resultado sin violar la semántica del programa.

La programación funcional pura tampoco implica la ausencia de entrada/salida o de estado. Haskell representa los efectos en el sistema de tipos y los secuencia mediante abstracciones como IO. De ese modo, el código puramente funcional permanece separado de las operaciones cuyo resultado depende del entorno. La ventaja conceptual no consiste en eliminar los efectos necesarios, sino en hacer explícito dónde ocurren y conservar el razonamiento ecuacional en el resto del programa (Marlow y Peyton Jones, 2012).

1.4. Tipos algebraicos y reconocimiento de patrones

Los tipos de datos algebraicos permiten construir nuevos tipos mediante alternativas y combinaciones de valores. Un tipo suma ofrece varios constructores posibles; un tipo producto reúne varios campos dentro de un mismo constructor. Ambas formas pueden combinarse y parametrizarse. El siguiente árbol binario, por ejemplo, puede estar vacío o contener un valor y dos subárboles:

```
data Arbol a
  = Vacio
  | Nodo a (Arbol a) (Arbol a)

tamano :: Arbol a -> Int
tamano Vacio = 0
tamano (Nodo _ izquierdo derecho) =
  1 + tamano izquierdo + tamano derecho
```

El parámetro *a* hace que la estructura sea polimórfica: el mismo tipo puede contener enteros, cadenas u otros valores. Los constructores *Vacio* y *Nodo* determinan las formas válidas del dato, mientras que el reconocimiento de patrones permite descomponerlas de manera declarativa. El sistema de tipos puede comprobar que las operaciones respeten esas formas y advertir, según las opciones del compilador, patrones faltantes o incompatibles.

Las listas de Haskell responden a la misma idea. Una lista es vacía o está formada por un elemento y otra lista. Esta definición recursiva es compacta, pero posee consecuencias operacionales: cada constructor puede convertirse en un objeto y sus componentes no necesariamente se evalúan al mismo tiempo. En una semántica no estricta es posible conocer la forma externa de una estructura sin haber calculado todavía todos sus campos. La diferencia entre evaluar un constructor y evaluar su contenido se formalizará en la unidad siguiente mediante las nociones de forma normal débil de cabeza (WHNF) y forma normal (NF).

1.5. Recursión y procesamiento de estructuras

En programación funcional, la repetición suele expresarse mediante recursión o mediante funciones de orden superior que encapsulan esquemas recursivos. La definición de tamaño anterior sigue directamente la estructura del árbol: el caso base corresponde a Vacío y el caso recursivo combina los resultados de ambos subárboles. Esta correspondencia entre la forma de los datos y la forma de las funciones facilita la construcción de definiciones composicionales y su análisis.

Operaciones como `map`, `filter`, `foldr` y `foldl` reúnen patrones de recorrido frecuentes. En lugar de reescribir la recursión para cada problema, el programador proporciona la operación específica y delega el recorrido a una abstracción reutilizable. Esto aumenta la expresividad, aunque el comportamiento espacial no puede deducirse únicamente de que una función sea recursiva o de orden superior: depende también del orden en que se demandan los resultados y de la estricticidad de los acumuladores.

Precaución conceptual. En Haskell, que una llamada esté sintácticamente en posición de cola no garantiza por sí solo un consumo constante de memoria. Un acumulador no estricto puede conservar una cadena de operaciones pendientes. La diferencia entre desbordamiento de `stack`, crecimiento del `heap` y `space leak` se desarrollará al estudiar los costos de la evaluación perezosa.

1.6. Haskell como lenguaje funcional no estricto

Haskell fue diseñado como un lenguaje funcional común para la investigación y se encuentra definido por el Haskell Report. Entre sus rasgos centrales se destacan la pureza, el tipado estático con inferencia, las funciones de orden superior, los tipos algebraicos y la semántica no estricta (Marlow y Peyton Jones, 2012). El compilador GHC constituye su implementación más difundida y reúne tres grandes componentes: el compilador propiamente dicho, las bibliotecas y el sistema de ejecución o RTS. La arquitectura interna de GHC se tratará en una unidad específica; aquí interesa solamente señalar que las propiedades del lenguaje condicionan el trabajo de esos componentes.

La no estricticidad establece que una función no necesita evaluar un argumento si su resultado no depende de él. Puede observarse con una función constante:

```
primero :: a -> b -> a
primero x _ = x

resultado = primero 1 undefined    -- produce 1
```

En un lenguaje con semántica estricta, los argumentos de la llamada se evalúan antes de aplicar la función; por lo tanto, el argumento `undefined` impediría obtener el resultado. En Haskell, `primero` no demanda su segundo parámetro y la expresión puede producir 1. El Haskell 2010 Language Report establece explícitamente que la aplicación de funciones es no estricta y ofrece mecanismos como `seq` o el operador `$!` para solicitar evaluación estricta cuando resulta conveniente (Marlow, 2010, sec. 6.2).

1.7. Semántica y estrategia de evaluación

Conviene distinguir la semántica del lenguaje de la estrategia utilizada para ejecutarlo. Los términos estricto y no estricto describen, en este contexto, propiedades semánticas: determinan qué resultados debe admitir el lenguaje y cómo se comporta una función frente a argumentos indefinidos. En cambio, `eager` y `lazy` describen estrategias operacionales: indican cuándo y de qué manera una implementación realiza los cálculos. Aunque es habitual asociar semántica estricta con evaluación `eager` y semántica no estricta con evaluación `lazy`, no se trata de pares lógicamente idénticos (Maessen, 2002, pp. 19-21).

GHC implementa la semántica no estricta de Haskell principalmente mediante evaluación por necesidad. Una expresión se suspende hasta que su valor es requerido; una vez calculada, el resultado puede compartirse para evitar repeticiones. Este mecanismo permite trabajar con estructuras potencialmente infinitas y separar productores de consumidores, pero introduce objetos suspendidos, actualizaciones y decisiones de retención que influyen directamente sobre el uso de memoria. La Unidad 2 desarrollará estos conceptos a partir de `thunks`, reducción de grafos, WHNF, NF y compartición.

1.8. Relación con la gestión de memoria

Las características presentadas tienen una traducción concreta en la implementación. Las funciones de orden superior requieren representar funciones junto con su entorno; la currificación hace necesario conservar aplicaciones parciales; los tipos algebraicos se materializan mediante constructores; la recursión crea y recorre estructuras enlazadas; y la no estricticidad obliga a representar cálculos aún no realizados. Pureza e inmutabilidad, por su parte, permiten compartir estructuras y ocultar ciertas actualizaciones internas sin modificar el significado observable del programa.

En consecuencia, analizar el consumo de memoria de Haskell exige vincular dos niveles. El nivel semántico explica qué resultados debe conservar el lenguaje y qué expresiones pueden permanecer sin

evaluar; el nivel operacional explica cómo GHC representa esa información en closures, thunks, stacks y objetos del heap. El resto del marco teórico avanzará desde el primero hacia el segundo, hasta disponer de los conceptos necesarios para interpretar los experimentos de asignación, residencia y retención de memoria de la PPS.

Alcance de la unidad. Este capítulo desarrolla la diferencia entre semántica estricta y no estricta, compara las principales estrategias de evaluación y explica la evaluación por necesidad mediante thunks, WHNF, forma normal, compartición y actualización. La representación concreta de estos mecanismos mediante closures y la máquina STG se reserva para la Unidad 3.

2. Estrategias de evaluación

La unidad anterior distinguió dos planos que conviene mantener separados durante todo el análisis. Por un lado, la semántica de un lenguaje determina qué valor denota un programa y qué transformaciones preservan su significado. Por otro, la estrategia de evaluación establece cómo una implementación selecciona y reduce las expresiones necesarias para obtener ese valor. En Haskell esta separación resulta especialmente importante: el lenguaje posee semántica no estricta, mientras que sus implementaciones habituales realizan esa semántica mediante evaluación perezosa o por necesidad (Maessen, 2002, pp. 18-24; Apfelmus, 2015).

La elección de una estrategia no es un detalle puramente abstracto. Determina cuándo se crea trabajo pendiente, cuándo debe ejecutarse, cuántas veces puede repetirse y qué información debe mantenerse viva mientras tanto. Estas decisiones explican por qué conceptos semánticos como la no estricticidad terminan teniendo consecuencias concretas sobre el heap, el stack y la residencia de memoria de un programa compilado con GHC.

2.1. Evaluación estricta y no estricta

Una primera distinción se refiere a la semántica. En un lenguaje estricto, una aplicación de función requiere valores para sus argumentos antes de producir el resultado de la aplicación. Esto se asocia naturalmente con call-by-value y con una ejecución eager: las subexpresiones se calculan antes de entrar en el cuerpo de la función. En un lenguaje no estricto, en cambio, el resultado puede estar definido aun cuando alguno de los argumentos no lo esté, siempre que ese argumento no sea necesario para determinar la respuesta (Maessen, 2002, pp. 19-20).

Esta diferencia puede formularse utilizando el valor semántico bottom, representado por $\perp$. Bottom modela un cómputo que no entrega un valor ordinario, por ejemplo porque no termina o porque produce un error. Una función f es estricta en un argumento cuando reemplazar ese argumento por $\perp$ hace que el resultado también sea $\perp$. Para una función unaria, la condición se expresa como $f \perp = \perp$. En funciones de varios parámetros, la estricticidad puede analizarse por separado para cada posición (Apfelmus, 2015).

El ejemplo clásico es una función constante. Su segundo argumento no participa en el resultado y, por lo tanto, Haskell puede devolver el primero sin evaluar el segundo:


```
constante :: a -> b -> a
constante x _ = x

resultado = constante 1 undefined    -- 1
```

El valor predefinido `undefined` representa una expresión que falla al ser evaluada. Sin embargo, `constante 1 undefined` produce `1` porque la función no demanda el segundo argumento. La observación importante no es solamente que una implementación pueda posponer ese cálculo, sino que la semántica de Haskell exige preservar un resultado compatible con esa no estricticidad. El Haskell Report define el lenguaje de manera no estricta sin imponer que toda implementación deba utilizar exactamente el mismo algoritmo de evaluación (Marlow, 2010; Apfelmus, 2015).

Distinción terminológica. No estricto y lazy no son sinónimos exactos. El primer término describe una propiedad del significado del lenguaje; el segundo describe una forma de ejecutar los programas. Mantener esta separación evita atribuir a la semántica detalles que en realidad pertenecen a la implementación.

2.2. Evaluación por necesidad

Una expresión puede contener varios redexes, es decir, subexpresiones que admiten un paso de reducción. Una estrategia de evaluación determina cuál de ellos se reduce primero. Entre las estrategias clásicas, `call-by-value` evalúa los argumentos antes de aplicar la función, mientras que `call-by-name` prioriza la aplicación exterior y pospone los argumentos hasta que el cuerpo los demanda. `Call-by-name` es relevante para los lenguajes no estrictos porque puede obtener una respuesta en casos donde una evaluación anticipada de un argumento divergiría; sin embargo, una sustitución ingenua puede repetir el mismo cómputo cada vez que el argumento aparece (Maessen, 2002, pp. 23-24).

La evaluación por necesidad, o `call-by-need`, conserva la capacidad de posponer cálculos de `call-by-name` pero agrega compartición. Una expresión se calcula solo cuando su valor es requerido; una vez calculada, el resultado se recuerda para que los usos posteriores no repitan el trabajo. Esta combinación de demanda y compartición constituye la idea operacional central de la evaluación perezosa utilizada en las implementaciones habituales de Haskell (Maessen, 2002, p. 23; Apfelmus, 2014).

Puede verse la diferencia con una función cuyo argumento aparece dos veces:

```

cuadrado :: Int -> Int
cuadrado x = x * x

-- Call-by-value (esquema conceptual)
cuadrado (1 + 2)
=> cuadrado 3
=> 3 * 3
=> 9

-- Call-by-name sin compartición
cuadrado (1 + 2)
=> (1 + 2) * (1 + 2)
=> 3 * (1 + 2)
=> 3 * 3
=> 9

-- Call-by-need: el argumento se comparte
cuadrado (1 + 2)
=> let x = 1 + 2 in x * x
=> let x = 3 in x * x
=> 9

```

En el último esquema, las dos apariciones de `x` no representan dos cálculos independientes: ambas remiten a la misma expresión compartida. Cuando esa expresión se demanda por primera vez, se evalúa y queda asociada al resultado 3. El segundo uso reutiliza ese resultado. Peyton Jones (1987) presenta esta idea mediante reducción de grafos, donde varias aristas pueden apuntar a un mismo subgrafo; Apfeldmus (2014) utiliza el mismo modelo para explicar por qué la compartición evita la duplicación de trabajo.

Sobre los ejemplos de reducción. Las secuencias anteriores son modelos conceptuales. Un programa real compilado con GHC atraviesa transformaciones y optimizaciones que pueden cambiar el orden y la forma concreta de las operaciones. El objetivo aquí es aislar las propiedades esenciales de cada estrategia, no reproducir instrucción por instrucción la ejecución del compilador.

2.3. Thunks, WHNF y forma normal

Para posponer un cómputo, una implementación lazy debe conservar información suficiente para realizarlo más adelante. Esa representación suspendida se denomina thunk. En un nivel abstracto, un thunk puede pensarse como una expresión no evaluada junto con el entorno necesario para recuperar sus variables libres. Cuando algún contexto necesita conocer el valor, el thunk es forzado y el cómputo continúa. En la Unidad 3 se precisará cómo GHC materializa estos objetos mediante closures; por ahora interesa su función dentro de call-by-need (Maessen, 2002, pp. 23-24).

Forzar un valor no implica necesariamente evaluarlo por completo. Los datos de Haskell pueden evaluarse por capas. Si una expresión produce un dato construido, con frecuencia basta con descubrir el constructor exterior para decidir cómo continuar, dejando sus campos sin evaluar. Este nivel mínimo se

denomina Weak Head Normal Form (WHNF). Para valores de datos, una expresión está en WHNF cuando se conoce su constructor más externo, aunque los argumentos del constructor todavía contengan thunks. Una función lambda también constituye WHNF porque ya es un valor que espera un argumento (Apfelmus, 2014; Wikibooks, 2026).

La forma normal, o Normal Form (NF), exige un grado mayor de evaluación: no deben quedar redexes pendientes en ninguna parte del valor finito. Por ello, estar en WHNF no garantiza estar en NF. La diferencia aparece claramente en tipos compuestos:

```
-- El constructor exterior ya es visible: WHNF, pero no NF
a = Just (1 + 2)

-- También está en WHNF: la cabeza y la cola pueden contener trabajo pendiente
b = (1 + 2) : ((3 + 4) : [])

-- Una vez evaluados todos los componentes, el valor está en NF
c = Just 3
```

En tipos simples como `Int`, alcanzar WHNF normalmente coincide con obtener el valor numérico y, por lo tanto, con NF. En una lista, un árbol o una tupla, en cambio, puede conocerse la estructura exterior mientras grandes partes del contenido permanecen suspendidas. Esta diferencia es fundamental para analizar memoria: decir que una expresión fue "evaluada" es ambiguo si no se especifica hasta qué profundidad.

El reconocimiento de patrones es una fuente habitual de demanda. Si una función debe distinguir entre `[]` y `(:)` para procesar una lista, necesita evaluar el argumento al menos hasta revelar el constructor exterior. No necesita, sin embargo, evaluar automáticamente los elementos de la lista. De forma similar, realizar reconocimiento de patrones sobre una tupla fuerza lo necesario para confirmar que el valor posee ese constructor, pero sus componentes pueden continuar como thunks. El tutorial de Wikibooks ilustra esta evaluación estratificada mediante pares y listas y muestra que diferentes contextos demandan diferentes profundidades (Wikibooks, 2026).

```
let z = (length [1..5], reverse "olleh")
    (n, s) = z
in n
```

Conceptualmente, `z` puede comenzar como una expresión suspendida. El patrón `(n, s)` necesita conocer que `z` es un par y por eso demanda su estructura exterior, pero no obliga a calcular inmediatamente `length [1..5]` ni `reverse "olleh"`. Cuando el resultado del bloque exige `n`, se fuerza el primer componente; `s` puede permanecer sin evaluar porque no contribuye al resultado. Este comportamiento resume la idea de "evaluar solo lo necesario" con más precisión que la afirmación informal de que Haskell simplemente retrasa todos los cálculos.

El combinador `seq` permite hacer explícita una demanda hasta WHNF: al evaluar `seq x y`, `x` debe alcanzar WHNF antes de continuar con `y`. Esta operación será retomada en la Unidad 5 al estudiar control de estricticidad y fugas de espacio. En esta etapa basta con notar que WHNF es precisamente el límite operacional que muchas operaciones de Haskell necesitan imponer sin forzar recursivamente una estructura completa (Apfelmus, 2014).

2.4. Compartición y actualización

La evaluación por necesidad requiere distinguir entre repetir una expresión y compartir su resultado. Una representación puramente textual puede sugerir que sustituir un parámetro duplica el código del argumento. La implementación lazy evita esa duplicación representando las expresiones como un grafo: si un valor se utiliza en varios lugares, todos ellos pueden conservar referencias al mismo nodo. Reducir ese nodo una vez hace que el resultado quede disponible para todos sus consumidores (Peyton Jones, 1987; Apfelmus, 2014).

La compartición por sí sola no basta. Cuando un thunk es forzado y finaliza su evaluación, la representación asociada debe actualizarse para registrar el resultado. De esta manera, un intento posterior de demandar el mismo valor no vuelve a ejecutar la expresión suspendida, sino que accede al valor ya obtenido. Maessen (2002, p. 24) describe este comportamiento como el reemplazo del thunk por su valor; en una implementación concreta también pueden intervenir indirecciones y otras formas de actualización que se desarrollarán al estudiar GHC.

```
let x = costoso 10
in x + x

-- 1. x referencia un único cómputo suspendido.
-- 2. El primer uso fuerza ese cómputo.
-- 3. El thunk se actualiza con el resultado.
-- 4. El segundo uso reutiliza el valor calculado.
```

Esta actualización introduce una forma de mutación interna que no contradice la pureza observable del lenguaje. Desde el punto de vista del programa, `x` sigue denotando el mismo valor y la transparencia referencial se conserva; lo que cambia es la representación utilizada por el runtime para evitar trabajo repetido. Esta separación entre significado puro e implementación mutable es una de las claves para comprender la arquitectura de los runtimes perezosos.

El mecanismo también tiene consecuencias espaciales. Mientras un thunk permanezca sin evaluar, debe conservar tanto la descripción del cómputo como las referencias que este necesita para ejecutarse. Después de forzarlo, ese conjunto puede ser reemplazado por una representación del resultado, que puede ocupar menos o más memoria que la expresión original. La compartición reduce trabajo duplicado, pero al mismo tiempo puede prolongar la vida de un objeto mientras exista algún

consumidor que mantenga una referencia hacia él. Por ello, el orden de evaluación y la estructura del grafo son relevantes tanto para el tiempo como para el espacio utilizado por un programa (Apfelmus, 2014).

Puente hacia la implementación. Hasta aquí thunk, compartición y actualización se describieron de manera abstracta. La Unidad 3 mostrará cómo GHC representa estos conceptos mediante closures, info tables y objetos del heap, y cómo la máquina STG organiza la entrada y actualización de los cálculos suspendidos.

Alcance de la unidad. Este capítulo conecta la evaluación por necesidad de la Unidad 2 con su implementación en el Glasgow Haskell Compiler (GHC). Se presenta el pipeline de compilación, el papel de Core, STG y C--, el modelo de ejecución de la máquina STG, la representación mediante closures y el tratamiento de aplicaciones currificadas. La organización detallada del heap y el garbage collector se desarrollan en la Unidad 4.

3. Implementación de Haskell en GHC

La unidad anterior describió la evaluación por necesidad en términos abstractos: una expresión puede permanecer suspendida, ser demandada más adelante y compartir el resultado de su evaluación. Para ejecutar eficientemente este modelo en una máquina convencional es necesario convertir esas ideas en estructuras concretas de código y datos. GHC resuelve este problema mediante una sucesión de lenguajes intermedios y un modelo de ejecución basado en la Spineless Tagless G-machine (STG). En ese recorrido, conceptos del nivel fuente como funciones, datos algebraicos y thunks adquieren una representación explícita que puede compilarse finalmente a código de bajo nivel (GHC Developers, 2026; Marlow y Peyton Jones, 2012).

Resulta útil separar dos aspectos. El primero es el pipeline del compilador: las transformaciones que llevan un módulo Haskell desde su sintaxis de alto nivel hasta representaciones cada vez más próximas al hardware. El segundo es el modelo de ejecución: las convenciones que determinan cómo se representan y se evalúan closures, cómo se pasan argumentos y cómo se continúa un cómputo suspendido. Ambos aspectos se encuentran en STG, que funciona como puente entre las optimizaciones funcionales de Core y la generación de código imperativo de bajo nivel.

3.1. Arquitectura y pipeline de compilación

GHC no traduce directamente el texto Haskell a instrucciones de máquina. El frontend primero analiza el programa, resuelve nombres y tipos y elimina gran parte del azúcar sintáctico. El resultado se expresa en Core, un lenguaje funcional intermedio pequeño y tipado. Sobre Core se ejecuta una parte importante de las optimizaciones de alto nivel, entre ellas simplificación, inlining, especialización, análisis de demanda y transformaciones worker/wrapper. Después de estas etapas, el programa se transforma a STG y posteriormente a C--/Cmm, representación de bajo nivel sobre la que se realizan transformaciones cercanas al control de flujo, stack y calling conventions. Desde Cmm los distintos backends pueden producir, por ejemplo, ensamblador nativo o LLVM (GHC Developers, 2026).

```
Código Haskell
  ↓ parsing, renaming, type checking, desugaring
Core
  ↓ optimizaciones Core-to-Core
STG
  ↓ representación explícita de la ejecución
C-- / Cmm
  ↓ backend
Código nativo / LLVM
```

Este pipeline no debe interpretarse como una única traducción lineal e inmutable. Cada representación posee sus propias pasadas y optimizaciones, y la versión concreta de GHC puede modificar el conjunto o el orden de algunas de ellas. Lo estable para el objetivo de este trabajo es la separación conceptual: Core conserva una forma funcional adecuada para razonar y optimizar expresiones; STG hace explícitos aspectos relevantes de la evaluación perezosa; y Cmm sirve como lenguaje de bajo nivel previo a la generación de código. La propia guía de GHC expone flags diferenciados para inspeccionar las etapas Core, STG y Cmm, reflejando esta división (GHC Developers, 2026).

Nivel de abstracción. El marco teórico no necesita describir cada pasada interna del compilador. El interés está en aquellas representaciones que permiten seguir el camino desde una expresión lazy hasta los objetos y operaciones que determinan su comportamiento temporal y espacial.

3.2. Core, STG y C--

Core es el lenguaje intermedio central de GHC para las optimizaciones funcionales. Aunque el código Haskell fuente puede contener una gran variedad de construcciones, Core utiliza un conjunto considerablemente menor de formas. Entre ellas aparecen variables, literales, aplicaciones, abstracciones lambda, bindings let y expresiones case. El desugaring convierte construcciones de superficie en estas formas más simples, de modo que las optimizaciones posteriores pueden operar sobre una representación uniforme. El sistema de tipos de Core conserva información rica, pero muchos detalles sintácticos del programa original dejan de ser relevantes una vez alcanzado este nivel (Marlow y Peyton Jones, 2012).

STG se encuentra más cerca de la ejecución. El lenguaje STG fue diseñado junto con una máquina abstracta para implementar lenguajes funcionales no estrictos sobre hardware convencional. Sus construcciones están elegidas para que tengan una lectura operacional directa: permiten distinguir funciones y thunks, identificar las variables libres que debe conservar una closure, expresar aplicaciones y hacer explícitos los puntos donde un valor debe evaluarse mediante case. Por esta razón, STG resulta especialmente útil para estudiar cómo una semántica call-by-need se transforma en asignación, entrada a closures y actualización (Peyton Jones, 1992; Marlow y Peyton Jones, 2004).

Una diferencia importante respecto del código fuente es que STG restringe la forma de las expresiones para simplificar su implementación. Los argumentos de una aplicación son átomos, y los bindings proporcionan la información necesaria para construir las closures correspondientes. Además, `case` constituye el mecanismo fundamental para demandar un valor: antes de seleccionar una alternativa, el scrutinee debe evaluarse lo suficiente como para descubrir su constructor o resultado exterior. Esto conecta directamente con el concepto de WHNF presentado en la Unidad 2 (Marlow, Rodríguez Yakushev y Peyton Jones, 2007).

```
-- Haskell (esquema)
f x = case x of
    Just y  -> y + 1
    Nothing -> 0

-- En STG, el case expresa el punto de demanda:
-- x debe revelar su constructor antes de elegir una alternativa.
```

La traducción desde STG hacia Cmm abandona progresivamente la apariencia funcional. Cmm puede considerarse una representación de bajo nivel, cercana a un ensamblador portable, pero todavía suficientemente estructurada para que el compilador gestione saltos, registros, stack y checks del runtime. En esta etapa aparecen de manera explícita operaciones como entrar a una closure, reservar espacio, comprobar límites del heap o del stack y transferir el control entre bloques de código. La Unidad 4 retomará estos mecanismos al estudiar el runtime system y la administración de memoria.

3.3. Máquina STG y representación mediante closures

La Spineless Tagless G-machine proporciona el modelo operacional histórico sobre el que se apoya la ejecución de GHC. La denominación *spineless* alude a que la máquina no mantiene la "spine" completa de una aplicación como una estructura de nodos de aplicación en el heap, sino que utiliza convenciones de llamada y stack para organizar los argumentos. Tagless, en su formulación original, refiere a que no es necesario inspeccionar primero una etiqueta explícita para decidir cómo evaluar un objeto: una closure puede ser "entrada" y su código asociado determina qué hacer. Las implementaciones modernas de GHC introducen pointer tagging como optimización, por lo que el nombre histórico no implica que la ejecución actual carezca por completo de tags (Peyton Jones, 1992; Marlow et al., 2007).

3.3.1. Registros y estado de ejecución

La máquina STG no describe únicamente objetos del heap: también modela un estado de ejecución que incluye registros abstractos para transportar argumentos y resultados y para mantener referencias al stack y a las áreas de asignación. Al descender hacia el código generado, GHC asigna esos registros virtuales a registros físicos de la arquitectura o a ubicaciones administradas por el runtime según la plataforma y la calling convention. Esta capa permite que valores de vida muy corta circulen sin

convertirse necesariamente en nuevas closures y explica por qué Cmm hace explícitas operaciones sobre registros, stack, heap y transferencias de control (Peyton Jones, 1992; GHC Developers, 2021).

Los registros también forman parte del límite entre ejecución y recolección de memoria: cuando el RTS toma el control, las referencias vivas conservadas en el estado de la máquina deben considerarse raíces junto con los stacks y los objetos estáticos. Las info tables y las convenciones del código generado permiten distinguir qué posiciones representan punteros y cuáles contienen datos no puntero. Por ello, registros, closures y frames no son mecanismos independientes, sino representaciones complementarias del estado que debe preservarse durante la ejecución (Yang, Marlow y Peyton Jones, 2013).

El objeto fundamental de este modelo es la closure. En GHC el término se utiliza de forma amplia para los objetos administrados que asocian una representación de datos con información sobre su comportamiento y layout. Conceptualmente, una closure del heap contiene un encabezado que permite localizar una info table y un payload. La info table describe el tipo de closure, el layout de su contenido y el código de entrada correspondiente; el payload contiene la información específica del objeto, como campos de un constructor o variables libres capturadas por una función o un thunk (Marlow et al., 2007; Yang et al., 2013).

Closure en el heap (modelo conceptual)

```
[ info pointer | payload ... ]
```

└─> Info table: tipo, layout, código de entrada, metadatos

Las variables libres explican por qué una función o un thunk necesita payload. Si una expresión suspendida utiliza valores definidos fuera de ella, esos valores deben continuar accesibles cuando el thunk sea demandado, aun si la ejecución ya abandonó el contexto donde se creó. Guardarlos en la closure proporciona el entorno necesario para reanudar el cómputo. Una closure de constructor, en cambio, usa su payload principalmente para almacenar los campos del dato. Esta representación uniforme permite que el runtime trate de manera sistemática objetos con comportamientos diferentes.

Las categorías de closure relevantes para este trabajo incluyen funciones, constructores ya evaluados, thunks, aplicaciones parciales (PAP) e indirecciones. Cuando se entra a una función, su entry code espera los argumentos según una convención determinada. Cuando se entra a un constructor ya evaluado, la ejecución puede devolver inmediatamente al contexto que demandó el valor. Cuando se entra a un thunk, en cambio, su código debe producir el valor suspendido y coordinar su actualización para que futuras demandas puedan reutilizar el resultado (Marlow et al., 2007).

Esta última operación es la materialización concreta del mecanismo de actualización explicado en la Unidad 2. Durante la evaluación de un thunk se instala información de continuación suficiente para

regresar al punto que necesita el resultado; una vez obtenido, el thunk puede actualizarse con el valor o quedar conectado a él mediante una indirección. Así, diferentes referencias al mismo thunk observan el mismo resultado sin recalculer el cuerpo. Los update frames y las variantes exactas de actualización pertenecen al runtime, pero su finalidad central es preservar la compartición de call-by-need (Peyton Jones, 1992; Marlow y Peyton Jones, 2004).

Closures e info tables. La info table es especialmente importante para la gestión de memoria: además del código de entrada y la clase de objeto, describe qué posiciones del payload contienen punteros. Esa información será utilizada por el garbage collector y se retomará en la Unidad 4.

3.4. Aplicación de funciones y modelo eval/apply

La currificación introduce otro problema de implementación. Desde el punto de vista semántico, una función de tipo $a \rightarrow b \rightarrow c$ puede verse como una función que recibe un argumento y devuelve otra función. Una implementación eficiente, sin embargo, no necesita crear literalmente una closure intermedia en cada llamada. Cuando el compilador conoce la función y su aridad, una aplicación saturada puede pasar directamente todos los argumentos mediante la calling convention correspondiente (Marlow y Peyton Jones, 2004).

La dificultad aparece cuando la función que será llamada no se conoce estáticamente. En ese caso deben compararse en tiempo de ejecución dos cantidades: la aridad de la función y el número de argumentos disponibles. Si coinciden, la llamada está saturada. Si hay menos argumentos que los requeridos, la aplicación está subsaturada y debe producirse una partial application o PAP que conserve la función y los argumentos ya recibidos. Si hay más, la aplicación está sobresaturada: se llama primero a la función con la cantidad que consume y los argumentos restantes se aplican al valor funcional obtenido (Marlow y Peyton Jones, 2004).

```
suma3 :: Int -> Int -> Int -> Int
suma3 a b c = a + b + c

suma3 1 2 3      -- aplicación saturada
suma3 1          -- subsaturada: produce una función/PAP
(f x) y z        -- puede requerir aplicar argumentos restantes al resultado
```

Históricamente se han utilizado dos modelos principales para resolver estas llamadas: push/enter y eval/apply. En push/enter, el llamador coloca los argumentos disponibles y entra a la función; el propio código de la función determina cuántos puede consumir. En eval/apply, en cambio, el llamador evalúa la closure funcional, consulta su aridad y elige cómo realizar la aplicación. Para una llamada desconocida, este último modelo centraliza el matching entre aridad y cantidad de argumentos en rutinas de aplicación y permite utilizar llamadas exactas cuando la información está disponible (Marlow y Peyton Jones, 2004).

Marlow y Peyton Jones implementaron y midieron ambos esquemas en GHC. Sus resultados mostraron costos de ejecución similares, con una ligera ventaja media para eval/apply en los experimentos del trabajo, pero la diferencia decisiva fue de ingeniería: eval/apply simplificaba el stack y facilitaba la traducción a representaciones portables de bajo nivel. Como consecuencia, GHC adoptó eval/apply. Este resultado es relevante para la PPS porque muestra que la representación de aplicaciones no es un detalle aislado: modifica la forma y frecuencia de PAPs, frames y accesos al runtime, y por tanto puede afectar tanto asignación como rendimiento.

La aplicación de funciones completa así el vínculo entre la abstracción funcional y la representación operacional. Una función de orden superior en Haskell puede terminar representada por una closure con aridad conocida o desconocida para el llamador; una aplicación incompleta puede convertirse en un objeto del heap; y demandar un resultado puede provocar entrada a closures y manipulación del stack. Para analizar el costo espacial de estos mecanismos resta describir cómo el RTS asigna, conserva y recupera esos objetos, tema de la siguiente unidad.

Puente hacia la gestión de memoria. La Unidad 3 identificó las estructuras que deben existir durante la ejecución: closures, info tables, payloads, PAPs y frames. La Unidad 4 estudiará dónde se almacenan, cómo se asigna espacio para ellas y cómo el garbage collector determina cuándo pueden dejar de ocupar memoria.

Alcance de la unidad. Este capítulo estudia dónde se almacenan las estructuras introducidas en la Unidad 3 y cómo el Runtime System (RTS) administra su vida útil. Se describen la organización del heap, el stack de los hilos Haskell, el garbage collector generacional y el ciclo de vida de los thunks. El diagnóstico de space leaks, las técnicas de forzado y las optimizaciones específicas de la evaluación perezosa se reservan para la Unidad 5.

4. Gestión de memoria en el Runtime System

La Unidad 3 mostró que la ejecución de Haskell en GHC depende de objetos concretos: closures de funciones y constructores, thunks, aplicaciones parciales e indirecciones, además de frames que representan continuaciones. Estas estructuras deben asignarse, mantenerse accesibles mientras puedan ser utilizadas y recuperarse cuando dejan de ser alcanzables. Esa tarea corresponde en gran medida al Runtime System (RTS), que acompaña al código generado por GHC y proporciona servicios de administración de memoria, garbage collection, stacks de los hilos Haskell, scheduling y soporte de ejecución (GHC Developers, 2026; Yang, Marlow y Peyton Jones, 2013).

En un lenguaje con evaluación perezosa, la gestión de memoria está estrechamente ligada al orden de evaluación. Una expresión que todavía no fue demandada puede existir como thunk en el heap y conservar en su payload las referencias necesarias para evaluarse más adelante. Cuando finalmente se fuerza, su representación puede cambiar para compartir el resultado. Por lo tanto, no basta con contar cuántos valores conceptuales produce un programa: también interesa qué closures se crean, qué referencias retienen y durante cuánto tiempo permanecen alcanzables. Esta relación será central al estudiar rendimiento y space leaks en la Unidad 5.

4.1. Organización del heap y asignación

El heap es el área donde el RTS administra la mayor parte de los objetos dinámicos de Haskell. Como se explicó en la Unidad 3, las closures poseen una representación uniforme basada en un encabezado y un payload. El encabezado permite localizar la info table asociada, que identifica la clase de objeto y describe el layout del payload, incluido qué campos deben interpretarse como punteros. Esa información no solo sirve al código de entrada de una closure: permite al garbage collector recorrer el grafo de objetos sin confundir punteros con datos no puntero (Yang et al., 2013; Marlow, Rodriguez Yakushev y Peyton Jones, 2007).

Heap (modelo conceptual)

```
[ closure A ] —> [ closure B ] —> [ closure D ]
      |
      +——> [ closure C ]
```

Cada info table describe qué posiciones del payload son referencias.

El heap del RTS se organiza en unidades de memoria denominadas blocks, que pueden agruparse en unidades mayores. Esta estructura evita exigir que todo el heap sea una única región física contigua y facilita reutilizar memoria, ampliar regiones de asignación y mover la pertenencia de bloques entre espacios o generaciones. Los objetos muy grandes requieren un tratamiento especial, pero la mayoría de las closures son pequeñas en relación con un bloque. Cada bloque mantiene además metadatos que permiten al RTS conocer, entre otros aspectos, su generación y su estado de ocupación (Yang et al., 2013).

La asignación ordinaria está diseñada para ser barata. Una región de la generación joven funciona como allocation area o nursery: el código generado mantiene un puntero hacia el siguiente espacio disponible y, para crear objetos consecutivos, puede avanzar ese puntero. Este esquema de bump allocation requiere muy pocas operaciones en el camino común. Antes de realizar determinadas asignaciones, el código ejecuta un heap check contra el límite disponible. Si no existe espacio suficiente, el control puede regresar al RTS para obtener más memoria o iniciar una recolección (Yang et al., 2013).

La nursery cumple además un papel generacional: allí nacen los objetos de vida corta. Esto resulta especialmente apropiado para programas funcionales, que suelen crear estructuras intermedias y closures temporales con alta frecuencia. El costo total de asignación puede ser elevado en bytes aun cuando la cantidad de memoria que permanece viva simultáneamente sea pequeña. Por ello es importante distinguir allocation (memoria acumulativamente solicitada durante la ejecución) de residency (memoria que continúa viva y alcanzable en un punto de observación). La guía del RTS reporta ambas magnitudes por separado en sus estadísticas (GHC Developers, 2026).

Asignar no equivale a retener. Un programa puede asignar muchas closures de vida muy breve y mantener baja residencia, mientras que otro puede asignar menos pero conservar un grafo alcanzable durante mucho tiempo. Esta diferencia será fundamental para interpretar los experimentos de memoria.

4.2. Stack y frames de continuación

El stack de GHC no debe identificarse sin más con la pila de llamadas convencional de C. En el modelo STG, el stack almacena principalmente continuaciones y otra información necesaria para reanudar la ejecución. Cuando una expresión case necesita evaluar su scrutinee, por ejemplo, el código puede colocar un frame que representa qué hacer con el resultado y luego entrar a la closure demandada. Cuando esa evaluación termina, el control regresa a la continuación situada en la parte superior del stack (Marlow et al., 2007).

Los frames también poseen información de layout utilizable por el garbage collector. De manera análoga a una closure del heap, una dirección de retorno puede asociarse a una info table que describe qué posiciones del frame contienen referencias. Esta propiedad permite recorrer con precisión los stacks al

buscar raíces: valores referenciados desde un frame activo no pueden recuperarse aunque no exista otra referencia a ellos en el heap (Yang et al., 2013; Marlow et al., 2007).

Stack de un hilo (esquema)

```
[ continuación más reciente ]  
[ update frame           ] —> thunk a actualizar  
[ continuación anterior   ]  
[ ...                     ]
```

Heap: closures y también los chunks que implementan el stack del hilo

Una clase particularmente relevante es el update frame. Cuando un thunk actualizable comienza a evaluarse, este frame conserva la información necesaria para que, al obtener el resultado, el thunk pueda modificarse y futuras demandas compartan ese valor. Así, el stack participa directamente en la implementación de call-by-need: no solo encadena retornos de funciones, sino que representa obligaciones de actualización asociadas a cómputos suspendidos (Peyton Jones, 1992; Yang et al., 2013).

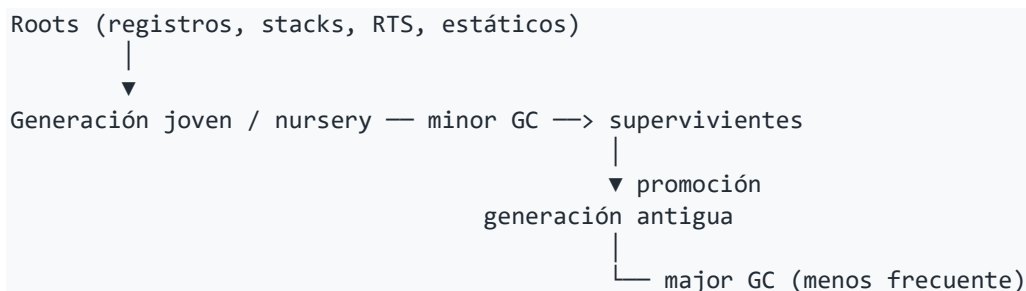
La implementación actual del RTS asigna los stacks de los hilos Haskell en el propio heap y permite que crezcan mediante chunks. La guía de GHC señala que un hilo comienza con un stack pequeño y añade nuevos chunks cuando los necesita; si luego dejan de ser necesarios, esos chunks adicionales pueden ser reclamados por el garbage collector. Esta estrategia contribuye a que los hilos Haskell tengan un costo inicial reducido y evita reservar anticipadamente una pila grande por cada hilo (GHC Developers, 2026).

La relación entre stack y heap también explica por qué ciertos patrones pueden manifestarse como problemas diferentes. Un cómputo puede consumir stack al acumular continuaciones pendientes, pero también puede mantener memoria en el heap mediante thunks o estructuras alcanzables desde esos frames. Reducir la profundidad de stack no garantiza por sí solo un uso espacial constante si persiste una cadena de evaluaciones suspendidas. Esta distinción es especialmente importante al razonar sobre recursión y acumuladores en Haskell perezoso.

4.3. Garbage collection generacional

La estrategia predeterminada de GHC es un garbage collector generacional de copia. El principio que motiva esta organización es la hipótesis generacional: los objetos recién creados tienen, en promedio, mayor probabilidad de volverse inalcanzables pronto que aquellos que ya sobrevivieron durante cierto tiempo. El heap se divide por ello en generaciones; los objetos nacen en la generación más joven y los supervivientes pueden permanecer en ella durante un período de envejecimiento o ser promovidos a generaciones más antiguas, que se recolectan con menor frecuencia (Yang et al., 2013; GHC Developers, 2026).

En una colección de copia, el collector parte de un conjunto de raíces y sigue las referencias para descubrir objetos alcanzables. Los objetos vivos de las regiones que se están recolectando son evacuados hacia el espacio de destino; luego el collector recorre los objetos evacuados para descubrir nuevas referencias, proceso conocido como scavenging. Los objetos que no son alcanzados desde las raíces quedan fuera del nuevo espacio vivo y su memoria puede reutilizarse. El uso de bloques permite reciclar eficientemente las regiones que ya fueron evacuadas (Yang et al., 2013).



Una colección menor se concentra en las generaciones jóvenes y, por lo tanto, puede ser mucho más barata que recorrer todo el heap. Una colección mayor incluye la generación antigua y permite medir con mayor fidelidad cuánta información de larga vida continúa siendo alcanzable. La configuración predeterminada documentada por GHC utiliza dos generaciones y permite modificar tanto su número como el tamaño del área de asignación mediante opciones del RTS; esos valores son parámetros de ejecución, no propiedades semánticas de Haskell (GHC Developers, 2026).

Para que una colección menor sea correcta también deben considerarse referencias que van desde objetos antiguos hacia objetos jóvenes. Como la generación antigua no se recorre por completo en cada minor GC, el RTS mantiene información sobre mutaciones relevantes mediante remembered sets o estructuras equivalentes. En el caso particular de los thunks, la actualización introduce una forma controlada de mutación: un thunk puede pasar de describir un cómputo pendiente a referenciar su resultado. Esta interacción obliga a coordinar actualización y promoción con el collector (Yang et al., 2013).

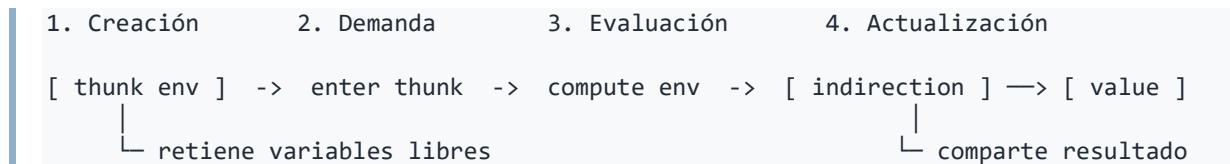
El collector de copia es la opción predeterminada, pero no es la única disponible. Las versiones actuales de GHC ofrecen también un non-moving collector concurrente de mark-and-sweep para la generación más antigua. Su objetivo principal es reducir las pausas de las major collections en determinadas cargas. Esta alternativa muestra por qué conviene separar el modelo general —alcanzabilidad, generaciones y reclamación— de una política concreta del RTS: varias estrategias pueden preservar la misma semántica observable con diferentes compromisos entre throughput, pausas y memoria (GHC Developers, 2026).

Alcance del recolector. Para esta PPS interesa principalmente comprender cómo la alcanzabilidad y las generaciones determinan la vida de las closures. Los detalles de GC paralelo o concurrente no son necesarios salvo que un experimento estudie explícitamente pausas o escalabilidad multinúcleo.

4.4. Representación y ciclo de vida de los thunks

Los thunks condensan la relación entre evaluación perezosa y memoria. Al crear un thunk, el programa reserva una closure cuyo payload conserva las variables libres necesarias para ejecutar el cómputo suspendido. Mientras exista una ruta desde las raíces hacia esa closure, tanto el thunk como las closures a las que apunta deben considerarse vivos, incluso si el resultado todavía no fue demandado. Por esta razón un thunk puede prolongar indirectamente la vida de una estructura mucho mayor que su propio tamaño (Peyton Jones, 1992; Yang et al., 2013).

Cuando un contexto demanda el valor, entra a la closure. Si el objeto es un thunk no evaluado, su entry code inicia el cómputo y establece el mecanismo de actualización correspondiente. El resultado se devuelve a la continuación y el thunk se modifica para que accesos posteriores puedan alcanzar el valor sin repetir la evaluación. En la descripción clásica de STG, esa actualización puede representarse mediante una indirección al resultado. Implementaciones y optimizaciones posteriores pueden eliminar o acortar algunas indirecciones, pero el principio de compartición sigue siendo el mismo (Marlow et al., 2007).



Desde la perspectiva del garbage collector, la actualización puede cambiar el grafo de referencias. Antes de evaluarse, el payload del thunk puede apuntar a varias variables libres. Después, una representación actualizada puede mantener únicamente la referencia necesaria hacia el resultado; las antiguas dependencias dejan de ser alcanzables a través de ese thunk y, si no poseen otras referencias, podrán ser recuperadas en una colección posterior. La evaluación no solo produce un valor: también transforma el grafo sobre el que opera el collector.

Sin embargo, el momento en que se produce esa transformación depende de la demanda. Un thunk que nunca es forzado puede conservar durante un período largo su entorno; una cadena de thunks puede retener estados intermedios o acumular trabajo pendiente. El garbage collector no puede liberar memoria que todavía sea alcanzable, aunque el programador ya no la considere conceptualmente útil. El collector determina alcanzabilidad, no intención. Esta diferencia es la base de muchas space leaks características de programas perezosos.

La evaluación concurrente agrega además la posibilidad de que más de un hilo demande el mismo thunk. GHC necesita coordinar ese estado para evitar problemas de reentrada y, cuando corresponde, duplicación innecesaria de trabajo. Los black holes son uno de los mecanismos utilizados con este fin. En

esta unidad basta señalar que el estado de un thunk puede evolucionar durante la ejecución; el funcionamiento y el costo de los black holes se desarrollarán en la Unidad 5 (Yang et al., 2013).

Puente hacia el rendimiento. La Unidad 5 partirá de esta distinción entre asignación y residencia para estudiar space leaks, retención accidental, mecanismos de estricticidad y optimizaciones como eval/apply, pointer tagging y black holes.

Alcance de la unidad. Este capítulo analiza las consecuencias de rendimiento del modelo descrito en las unidades anteriores. Se explicitan primero los costos temporales y espaciales de la evaluación perezosa; luego se estudian space leaks y retención, mecanismos de estricticidad y forzado, eval/apply y dos optimizaciones del RTS estrechamente vinculadas con la pereza: dynamic pointer tagging y black holes. Las estrategias alternativas eager/lazy se reservan para la Unidad 6.

5. Rendimiento de la evaluación perezosa

Las unidades anteriores mostraron que la pereza no es una propiedad sin costo: para posponer un cálculo, GHC debe representar trabajo pendiente mediante thunks; cuando ese trabajo se demanda, debe entrar a la closure, evaluar su cuerpo y actualizarla para compartir el resultado. A la vez, el garbage collector solo puede recuperar objetos que hayan dejado de ser alcanzables. Por ello, el rendimiento de un programa Haskell depende no solo del algoritmo expresado en el código fuente, sino también del orden efectivo de evaluación, de las closures creadas y del grafo de referencias que permanece vivo durante la ejecución (Marlow, Rodríguez Yakushev y Peyton Jones, 2007).

Esta relación produce un compromiso característico. La evaluación por necesidad evita computar valores que nunca se utilizan y permite compartir resultados, pero también introduce asignaciones, saltos indirectos, actualizaciones y posibilidades de retención que no aparecen de la misma forma en una ejecución estricta. GHC compensa parte de estos costos mediante análisis del compilador y optimizaciones del RTS. Sin embargo, ciertos patrones dependen de la demanda dinámica del programa y siguen requiriendo que el programador observe y controle cuándo se fuerza una estructura.

5.1. Costos temporales y espaciales de la evaluación perezosa

Representar un cómputo diferido tiene un costo incluso antes de ejecutarlo. Crear un thunk suele requerir asignar una closure y conservar en su payload las variables libres necesarias para reanudar la expresión. Cuando el valor se demanda, la ejecución debe entrar al código asociado a la closure, determinar la acción correspondiente y, si se trata de un thunk actualizable, producir el resultado y registrarlo para que las demandas posteriores no repitan el trabajo. La compartición evita recomputación, pero introduce a cambio creación de objetos, entradas a closures, escrituras de actualización y, en ciertos casos, direcciones adicionales (Peyton Jones, 1992; Marlow y Peyton Jones, 2004).

Parte del costo temporal aparece en el flujo de control. Cuando el tipo o el estado de una closure no se conoce estáticamente, entrar a ella puede implicar un salto indirecto; estos saltos son más difíciles de predecir que una llamada directa y pueden provocar fallos de predicción. Dynamic pointer tagging, estudiado más adelante, fue diseñado precisamente para evitar muchas entradas a constructores ya evaluados. A esto se suma el trabajo del garbage collector: una tasa alta de asignación obliga a recorrer,

copiar o marcar objetos con mayor frecuencia, y una residencia elevada aumenta el conjunto vivo que debe procesarse durante algunas colecciones (Marlow, Rodriguez Yakushev y Peyton Jones, 2007; Yang, Marlow y Peyton Jones, 2013).

El costo espacial es igualmente dependiente de la demanda. Los thunks ocupan memoria y pueden retener transitivamente estructuras mucho mayores que su propio tamaño; cadenas de operaciones pendientes, acumuladores perezosos o referencias conservadas por una closure pueden prolongar la vida de objetos que de otro modo quedarían fuera del grafo alcanzable. Por eso allocation y residency deben medirse por separado: el primero cuantifica cuánto se asigna a lo largo de la ejecución, mientras que el segundo describe cuánto de ese material continúa vivo en un momento dado. La subsección siguiente estudia el caso patológico en el que esa retención se vuelve una space leak.

Lectura de costos. No todo valor perezoso paga todos estos costos ni una ejecución estricta los elimina sin contrapartidas. La magnitud depende de qué closures crea el compilador, qué valores son demandados, cuánto logra optimizar GHC y qué objetos sobreviven a las colecciones.

5.2. Space leaks, retención y diagnóstico

Una space leak ocurre cuando un programa conserva más memoria de la necesaria para realizar su cómputo, aun cuando esa memoria siga siendo correctamente alcanzable para el garbage collector. No equivale, por lo tanto, al caso clásico de memoria perdida por falta de liberación manual. En un lenguaje recolectado, el collector cumple su contrato al mantener todo objeto alcanzable; el problema aparece cuando una referencia o un thunk prolonga innecesariamente la vida de datos que, desde la intención del programa, podrían haberse descartado antes (Mitchell, 2013).

La pereza puede amplificar esa retención porque un thunk conserva en su entorno las variables libres necesarias para ejecutarse. Si una expresión representa una secuencia de transformaciones aún no demandadas, el heap puede contener no solo el resultado conceptual sino también los argumentos y estados intermedios requeridos para obtenerlo. Un thunk pequeño puede así mantener vivo un subgrafo grande. Evaluarlo modifica el grafo y puede eliminar esas referencias, permitiendo que una colección posterior recupere memoria, tal como se anticipó al describir el ciclo de vida de los thunks en la Unidad 4.

Antes de forzar:

[thunk: delete dead set]



dead



set completo

Después de forzar:

[resultado]



set útil

El GC no puede liberar "dead" mientras siga alcanzable desde el thunk.

El caso documentado por Mitchell (2013) en el sistema de build Shake ilustra este fenómeno. Un conjunto de identificadores de hilos era actualizado de manera perezosa. Aunque cada hilo terminado se eliminaba lógicamente del conjunto, las operaciones pendientes seguían reteniendo sus ThreadId y, transitivamente, sus stacks. El síntoma era un crecimiento de memoria mucho mayor que el tamaño aparente de la estructura. Forzar el campo correspondiente mediante estricticidad redujo la memoria del caso real aproximadamente en un gigabyte. El ejemplo muestra que la magnitud de una space leak no puede deducirse únicamente del objeto que origina la referencia.

Otro patrón frecuente aparece en acumuladores y contenedores. Una estructura puede ser estricta en su spine pero mantener valores perezosos; aplicar repetidamente una función no estricta puede acumular thunks dentro de los elementos. Yang (2011) muestra que agregar un bang pattern al argumento de la función combinadora no siempre fuerza el resultado en el punto adecuado: a veces la estricticidad debe incorporarse a la operación que construye el contenedor, por ejemplo mediante una variante estricta. Esto refuerza una idea central: la pregunta relevante es qué closure se fuerza y en qué momento, no simplemente si aparece una anotación de estricticidad en algún lugar.

El diagnóstico requiere separar allocation de residency. Una tasa alta de asignación puede ser compatible con baja residencia si los objetos mueren rápidamente; una space leak, en cambio, suele manifestarse como crecimiento de memoria viva. Las estadísticas del RTS y el heap profiling permiten observar la evolución del heap y clasificar objetos por criterios como tipo o descriptor; el retainer profiling puede ayudar a identificar qué objetos mantienen viva una región. En la práctica también es valioso reducir el programa hasta un caso reproducible, porque la pereza puede separar considerablemente el lugar donde se crea una estructura del lugar donde finalmente se demanda (Mitchell, 2013).

El GC no corrige una space leak. Si un objeto continúa alcanzable a través de un thunk o una estructura viva, recolectarlo sería incorrecto. La solución consiste en modificar la demanda o las referencias retenidas, no en pedir al collector que libere objetos todavía accesibles.

5.3. Estricticidad y forzado como herramientas de control

Haskell permite introducir estricticidad de forma localizada sin abandonar su semántica no estricta general. El operador seq evalúa su primer argumento hasta weak head normal form (WHNF) antes de devolver el segundo; la aplicación estricta \$! usa esa idea para forzar el argumento antes de aplicarlo. Los bang patterns permiten expresar puntos de forzado de manera sintáctica y los campos estrictos trasladan esa decisión a la construcción de un tipo de datos. Para estructuras que deben evaluarse por completo, deepseq y la clase NFData permiten avanzar desde WHNF hacia normal form (NF).

```
seq x y      -- fuerza x hasta WHNF antes de producir y
f $! x       -- aplicación estricta: fuerza x hasta WHNF
let !x = e    -- bang pattern: demanda e en este punto
foldl' f z xs -- fuerza el acumulador en cada paso (hasta WHNF)
```

La distinción entre WHNF y NF impide interpretar estos mecanismos como un interruptor binario entre lazy y eager. Forzar una lista a WHNF solo revela si su constructor externo es vacío o cons; no exige evaluar ni la cola completa ni el elemento contenido. Del mismo modo, un registro puede estar en WHNF y contener campos que todavía son thunks. Si la retención se encuentra dentro de una estructura anidada, seq puede ser insuficiente; si solo interesa exponer el constructor externo, deepseq puede realizar trabajo innecesario. El nivel de evaluación debe elegirse de acuerdo con el grafo que realmente necesita dejar de retenerse.

Un ejemplo clásico es foldl. En una operación como una suma, el acumulador perezoso puede representar una cadena de aplicaciones pendientes en lugar de un número ya calculado. foldl' fuerza el acumulador en cada iteración y evita esa cadena cuando WHNF es suficiente para el tipo del acumulador. Esta diferencia también explica por qué una función sintácticamente tail-recursive no garantiza por sí sola uso espacial constante en Haskell: reutilizar la continuación no elimina thunks que se hayan acumulado en el heap.

GHC realiza además análisis de demanda y estricticidad para descubrir evaluaciones que pueden adelantarse sin cambiar el resultado observable. Esa información permite eliminar algunas suspensiones, evitar ciertas asignaciones o trabajar con valores no boxed en caminos optimizados. Sin embargo, la información estática no sustituye la medición: agregar estricticidad indiscriminadamente puede ejecutar cálculos que nunca serían demandados, reducir la productividad de estructuras infinitas o incrementar la residencia al producir datos antes de que el consumidor los necesite. El objetivo es hacer más predecible la vida de los objetos en los puntos donde el perfil demuestra que la pereza resulta costosa.

5.4. Eval/apply y el costo de la aplicación de funciones

La currificación introduce otra fuente de decisiones dinámicas. Cuando el compilador conoce la función y su aridad, una llamada saturada puede utilizar una convención directa y eficiente. El problema aparece en llamadas de orden superior donde la función concreta no se conoce estáticamente: en tiempo de ejecución deben combinarse la aridad de la closure y la cantidad de argumentos suministrados. Las estrategias push/enter y eval/apply difieren en quién realiza esa conciliación (Marlow y Peyton Jones, 2004).

Llamada a función de aridad desconocida

```
push/enter : caller pone argumentos -> función decide cuántos consumir
eval/apply : caller evalúa la closure -> consulta aridad -> llama / crea PAP /
reaplica
```

En eval/apply, el caller evalúa primero la closure de función y consulta su aridad. Si coincide con la cantidad de argumentos, realiza la llamada exacta; si faltan argumentos, construye una partial application (PAP); si sobran, aplica la función a los argumentos que consume y continúa aplicando el resultado. Este modelo simplifica la organización del stack y el mapeo hacia lenguajes intermedios de bajo nivel. Los experimentos originales dentro de GHC encontraron costos de ejecución muy similares entre ambos modelos, con una ligera ventaja media para eval/apply y ventajas claras de complejidad de implementación (Marlow y Peyton Jones, 2004).

Para esta PPS, eval/apply interesa porque muestra que el costo de la pereza no se reduce a crear thunks. La representación de funciones, PAPs, argumentos y frames también determina asignaciones y accesos en memoria. Aun cuando dos modelos preserven la misma semántica de alto nivel, pueden distribuir de manera diferente el trabajo entre caller, closure y RTS. El rendimiento debe analizarse, por ello, sobre la representación operacional y no únicamente sobre la expresión Haskell original.

5.5. Dynamic pointer tagging

La máquina STG clásica fue concebida como tagless: para descubrir qué representa una closure, el código puede entrar a ella y dejar que su entry code actúe según se trate de un thunk, una función o un valor ya evaluado. Esta uniformidad simplifica el modelo, pero entrar a una closure ya evaluada puede introducir saltos indirectos difíciles de predecir. Marlow, Rodríguez Yakushev y Peyton Jones (2007) estudiaron este costo y propusieron explotar información de tags para evitar entradas innecesarias.

Dynamic pointer tagging utiliza los bits menos significativos que quedan disponibles por la alineación de los punteros. En la representación estudiada, un puntero de 32 bits dispone de dos bits bajos y uno de 64 bits de tres. Esos bits pueden codificar si el objeto apuntado es un constructor y, cuando el número de constructores lo permite, cuál es su tag. Un valor no nulo proporciona información segura; cero conserva el caso general en el que el código debe inspeccionar o entrar a la closure.

```
Puntero alineado:      ...xxxxxxxx000
Pointer tagging:       ...xxxxxxxxTTT
```

```
TTT = información rápida sobre evaluatedness / constructor
Antes de desreferenciar la dirección real, el RTS elimina los bits de tag.
```

La ventaja no proviene de comprimir significativamente el heap, sino de evitar parte del tráfico de control y de memoria necesario para descubrir el estado del objeto. El paper reporta reducciones medias de tiempo de ejecución del 13,7 % en una máquina AMD Opteron de 64 bits y del 12,8 % en un

Intel de 32 bits, a cambio de un aumento de algo más del 2 % en tamaño de binario. Las mediciones relacionan buena parte de la mejora con la reducción de branch mispredictions (Marlow et al., 2007). Estos números corresponden a la plataforma y versión estudiadas y no deben extrapolarse como una garantía de rendimiento para GHC actual.

El costo de la técnica es su carácter transversal. Un puntero tagged no puede desreferenciarse como una dirección ordinaria sin retirar antes los bits auxiliares; por ello compilador, garbage collector y RTS deben respetar la convención y propagar correctamente los tags cuando mueven o actualizan objetos. La optimización constituye un ejemplo directo de cómo una decisión de representación a nivel de bits puede reducir el overhead asociado a la evaluación perezosa sin modificar la semántica del programa.

5.6. Black holes y evaluación concurrente de thunks

Cuando varios hilos demandan una misma suspensión aparece un problema adicional: si el thunk tarda en evaluarse, repetir el cómputo en paralelo puede desperdiciar trabajo y determinadas formas de reentrada pueden formar ciclos. GHC utiliza black holes para representar thunks que se encuentran bajo evaluación. Conceptualmente, reclamar un thunk consiste en sustituir temporalmente su estado por una representación que permite detectar que otro cómputo ya está trabajando sobre él; los hilos competidores pueden bloquearse y ser despertados cuando la actualización produce el valor (Yang, Marlow y Peyton Jones, 2013).

```
[ thunk ] --demanda--> [ BLACKHOLE / en evaluación ] --resultado--> [ value ]
                        ^
                        |
                        └─ otra demanda: detectar competencia / esperar
```

El RTS distingue estrategias con diferentes costos de sincronización. El llamado eager blackholing puede marcar tempranamente un thunk en un camino donde introducir sincronización fuerte sería caro, por lo que no necesariamente garantiza exclusividad ante una carrera. El lazy blackholing se realiza con menor frecuencia, cuando el hilo se desprograma mientras continúa evaluando, y puede utilizar una operación atómica compare-and-swap para reclamar el thunk. El diseño busca que la ejecución común de thunks no pague continuamente el precio de la sincronización, reservándola para los casos donde la competencia se vuelve relevante (Yang et al., 2013).

Los black holes también muestran que la compartición prometida por call-by-need tiene una dimensión de implementación. En un modelo secuencial basta razonar con que un thunk se actualiza después de evaluarse; en un runtime concurrente es necesario coordinar quién realiza la evaluación y qué observan los demás hilos durante esa transición. Estas decisiones influyen en trabajo duplicado, bloqueos y escritura del heap, aun cuando el valor funcional final sea el mismo.

Síntesis. Space leaks, estricticidad, eval/apply, pointer tagging y black holes actúan en niveles distintos, pero comparten un eje: el momento y la representación de la evaluación determinan qué objetos se crean, cuáles permanecen vivos y qué trabajo realiza el RTS. La Unidad 6 utilizará este marco para comparar estrategias alternativas eager/lazy y vincularlas con los experimentos de la PPS.

Alcance de la unidad. Este capítulo cierra el marco teórico estudiando una alternativa a la ejecución puramente perezosa: la evaluación híbrida eager/lazy propuesta en Eager Haskell. Se describe la estrategia resource-bounded, sus mecanismos de suspensión y fallback, sus ventajas y costos, y finalmente se vinculan estos conceptos con los tres experimentos definidos para la PPS. El objetivo no es establecer una estrategia universalmente superior, sino formular qué efectos observables permiten compararlas.

6. Estrategias alternativas y trabajo relacionado

El recorrido anterior mostró que la semántica no estricta de Haskell y su estrategia de ejecución deben distinguirse. La implementación tradicional mediante call-by-need garantiza que una expresión se evalúe solo cuando su valor es demandado y que el resultado se comparta. Esta decisión hace posibles estructuras potencialmente infinitas y evita trabajo innecesario, pero exige thunks, actualizaciones y una gestión cuidadosa del heap. En el extremo opuesto, call-by-value ejecuta los argumentos antes de la aplicación y ofrece un flujo de control más próximo al de los lenguajes estrictos, aunque no constituye por sí mismo una estrategia normalizante para un lenguaje no estricto (Maessen, 2002).

Esta diferencia abre una pregunta relevante para la PPS: cuánto del costo observado proviene de la semántica funcional y cuánto de la estrategia concreta utilizada para hacerla ejecutable. Eager Haskell, desarrollado por Maessen, explora precisamente ese espacio. Su propuesta conserva la semántica ecuacional de Haskell, pero intenta ejecutar eager el caso común y recurre a suspensiones cuando continuar en ese orden amenaza con consumir recursos sin producir una respuesta necesaria. La comparación con GHC resulta así conceptual y experimental: son implementaciones distintas que preservan el mismo lenguaje fuente mediante decisiones operacionales diferentes.

6.1. Evaluación híbrida eager/lazy

6.1.1. Motivación: pereza, eagerness y recursión

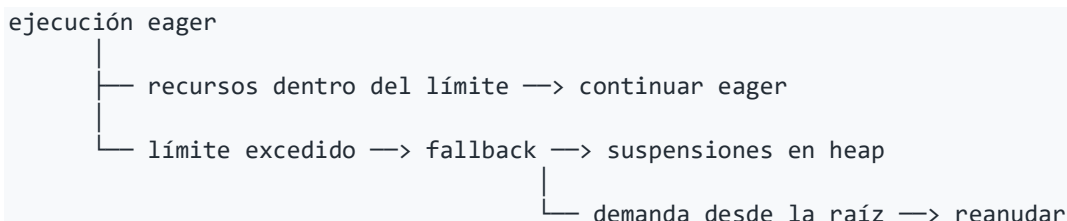
En una implementación lazy, un acumulador recursivo puede convertirse en una cadena de thunks si su valor no se demanda en cada iteración. La Unidad 5 mostró el ejemplo de foldl frente a foldl': que la llamada sea tail-recursive no basta para garantizar uso espacial constante si el acumulador permanece perezoso. Un compilador puede descubrir estricticidad y adelantar parte de ese trabajo, o el programador puede introducir seq, \$! o bang patterns, pero el comportamiento espacial depende entonces del análisis y de los puntos explícitos de forzado.

Maessen (2002) toma este problema como una motivación central. En una ejecución eager, el acumulador se calcula normalmente antes de la siguiente iteración y la recursión de cola puede comportarse como un bucle sin construir una cadena equivalente de suspensiones. El inconveniente es

que ejecutar eager de manera irrestricta cambiaría el comportamiento de programas válidos que contienen trabajo no demandado, divergencia o estructuras infinitas. La propuesta híbrida intenta conservar ambas propiedades: favorecer el camino eager cuando progresa de forma acotada y recuperar evaluación bajo demanda cuando ese camino deja de ser seguro.

6.1.2. Resource-bounded hybrid evaluation

La estrategia de Eager Haskell ejecuta ordinariamente las computaciones en orden de programa, como lo haría una implementación estricta. Sin embargo, la ejecución está sujeta a límites de recursos. Cuando se excede un límite, el runtime señala una excepción interna e inicia fallback: deja de permitir que el stack crezca y convierte las llamadas y computaciones pendientes en suspensiones almacenables. Una vez que el stack se ha desenrollado, la ejecución vuelve a partir de la raíz; las suspensiones existentes solo se reanudan cuando el resultado que representan es efectivamente demandado (Maessen, 2002, cap. 4).



El modelo considera límites sobre stack y espacio total; además necesita evitar que una computación consuma tiempo ilimitado sin asignar de una forma directamente observable. En la formalización de la tesis, el progreso del heap se utiliza como parte del mecanismo para aproximar y reajustar el presupuesto de recursos, en particular después de una recolección. No debe interpretarse esto como una política vigente de GHC: es una decisión del runtime experimental de Eager Haskell destinada a acotar cuánto trabajo puede avanzar fuera de la demanda proveniente de la raíz.

La suspensión es el elemento que permite reconciliar la ejecución eager con la semántica no estricta. El trabajo iniciado puede materializarse como un objeto reanudable; si ninguna ruta desde el resultado final lo demanda, esa suspensión puede dejar de ser alcanzable y ser recuperada por el garbage collector. Si sí resulta necesaria, se fuerza de manera análoga a un thunk. De este modo la estrategia no elimina la pereza, sino que desplaza el momento en que se crea trabajo suspendido.

Distinción clave. Eager Haskell no equivale a compilar Haskell como un lenguaje estricto. Su ejecución preferentemente eager está subordinada a mecanismos de suspensión y fallback que preservan el comportamiento no estricto. La tesis presenta esta equivalencia semántica como una propiedad fundamental de la estrategia híbrida.

6.1.3. Consecuencias sobre stack, heap y garbage collection

La diferencia de estrategia modifica dónde vive el estado de una computación. En el caso eager común, Eager Haskell utiliza un call stack convencional para agrupar bindings y almacenar estado local. Cuando debe suspenderse trabajo, el estado necesario pasa a representarse en el heap. Esta separación busca aprovechar la localidad y el bajo costo del stack cuando la ejecución es lineal, sin renunciar a objetos de vida arbitraria cuando la no estricticidad exige conservar una computación (Maessen, 2002, caps. 4-5).

El cambio tiene efectos directos sobre allocation y residency. Ejecutar un valor tempranamente puede evitar construir un thunk que después habría que forzar, pero también puede producir datos que finalmente no sean utilizados. Suspender durante fallback introduce a su vez objetos y trabajo de reanudación. La estrategia híbrida no hace desaparecer el costo de memoria: intenta modificar qué se asigna y durante cuánto tiempo, manteniendo acotado el trabajo especulativo. Por ello allocation, memoria residente y tiempo de GC deben observarse por separado al comparar políticas.

La tesis incorpora además decisiones específicas de runtime, como un collector generacional que usa información del stack como predictor adicional de vida de los objetos. Este detalle muestra que evaluación y memoria no son capas independientes: cambiar la representación de trabajo pendiente puede alterar también qué información dispone el recolector. Para la PPS, sin embargo, esta optimización se considera trabajo relacionado y no una característica a reproducir en GHC.

6.1.4. Evidencia experimental y límites de la propuesta

Los resultados de Eager Haskell son deliberadamente mixtos y deben leerse como evidencia histórica de un prototipo, no como una comparación actual entre compiladores. En los benchmarks de la tesis, la recursión con acumuladores muestra el comportamiento que motivó la propuesta. Para una variante del benchmark multiplier, eliminar anotaciones de estricticidad ralentizó aproximadamente un 20 % el código optimizado por GHC, mientras que en Eager Haskell esas anotaciones no produjeron un efecto medible. En una versión de Queens intensiva en tail recursion, Eager Haskell llegó a superar a GHC en el escenario estudiado (Maessen, 2002).

Al mismo tiempo, el prototipo fue en promedio más lento que el GHC utilizado por el autor: la tesis reporta un slowdown geométrico cercano al 60 % en su conjunto de benchmarks, y wavefront exhibe un caso extremo ligado a fallback, suspensiones y garbage collection. Multiplier también resulta costoso porque genera numerosas listas infinitas de las que finalmente se consume una parte importante; allí crear suspensiones para luego forzarlas puede competir desfavorablemente con ejecutar directamente el trabajo. Estas mediciones fueron realizadas sobre hardware y versiones de compiladores de comienzos de los años 2000, por lo que su valor para esta investigación es explicativo, no predictivo.

La lección relevante es que no existe una dicotomía simple donde eager reduzca siempre memoria y lazy evite siempre trabajo. El resultado depende del patrón de demanda. Si casi todo valor producido se consume, adelantar su evaluación puede reducir thunks y actualizaciones. Si grandes porciones se descartan o la estructura es intrínsecamente infinita, la pereza evita trabajo. Una estrategia híbrida intenta adaptarse entre ambos casos, pero paga por detectar límites, suspender, reanudar y coordinar esas transiciones.

6.2. Relación con los experimentos de la PPS

Los experimentos definidos para la PPS no implementan Eager Haskell ni modifican el RTS de GHC. Su objetivo es más acotado: observar cómo decisiones de evaluación y representación se manifiestan en las etapas intermedias, en el uso de memoria y en el código de bajo nivel. La evaluación híbrida funciona aquí como trabajo relacionado y como contraste conceptual que ayuda a formular hipótesis, sin ampliar el protocolo experimental previamente establecido.

6.2.1. Experimento 1: factorial, Core y STG

El primer experimento compila factorial en Haskell y examina los dumps de Core y STG. El interés está en seguir las transformaciones del programa, la información que el compilador puede utilizar para optimizarlo y la representación operacional mediante closures. Un factorial con acumulador es particularmente útil porque conecta con el problema discutido por Maessen: la forma fuente puede ser tail-recursive, pero su comportamiento espacial depende de si el acumulador se mantiene suspendido o puede ser evaluado de manera estricta.

Core permite observar qué transformaciones y análisis han cambiado la expresión antes de llegar al modelo de ejecución; STG aproxima la observación a closures, llamadas y evaluación. La hipótesis cualitativa no es que deba aparecer una forma interna determinada, porque esta depende de la versión y de las optimizaciones de GHC, sino que las variantes que inducen mayor estricticidad deberían hacer visible una menor necesidad de representar trabajo diferido. El dump será la evidencia, no el código fuente por sí solo.

6.2.2. Experimento 2: heap profiling y estricticidad

El segundo experimento mide memoria con factorial, map y la comparación foldl frente a foldl'. Aquí el vínculo con las Unidades 4 y 5 es directo: se observarán crecimiento del heap, retención y space leaks mediante heap profiling. La comparación lazy/strict permite comprobar empíricamente que forzar el acumulador puede acortar la vida de cadenas de thunks, pero también obliga a distinguir allocation de residency y trabajo del garbage collector.

Programa / variante	-> evidencia principal
factorial	-> acumulador, closures y evolución del heap
map	-> producción/consumo perezoso de una estructura
foldl vs foldl'	-> thunks acumulados vs forzado del acumulador
Perfiles RTS	-> allocation, residency, retención y trabajo de GC

La evaluación híbrida aporta una interpretación adicional: si el costo proviene de conservar trabajo pendiente que finalmente será consumido, adelantar evaluación puede ser ventajoso; si el consumidor ignora parte de la estructura, hacerlo podría aumentar el trabajo. Por eso los gráficos y la tabla de resultados deben describir la forma observada del heap y no limitarse a un único valor máximo de memoria.

6.2.3. Experimento 3: Haskell, C e IR de bajo nivel

El tercer experimento utiliza casos equivalentes en Haskell y C y observa el LLVM IR y las instrucciones generadas, con foco en el uso observable de stack y heap. No se pretende demostrar que una abstracción de Haskell corresponda uno a uno con una instrucción o asignación en C; el propósito es identificar cómo las distintas estrategias y runtimes materializan estado de ejecución, llamadas y objetos intermedios. LLVM/Clang y los dumps de GHC proporcionan niveles complementarios de observación.

Este experimento completa el hilo del marco teórico: desde la semántica no estricta de la Unidad 2 se pasó por Core/STG y closures, por heap y garbage collection, y por los costos de la pereza. La comparación con C permite volver a una ejecución donde stack, llamadas y temporales son más explícitos, mientras que Eager Haskell demuestra como trabajo relacionado que incluso dentro de un lenguaje no estricto existe margen para redistribuir estado entre stack, heap y suspensiones sin abandonar la semántica del lenguaje.

6.2.4. Criterios de comparación y cierre

Los tres experimentos deben interpretarse con un conjunto común de criterios: predictibilidad del uso de memoria, overhead de la representación, control sobre la vida útil de los objetos, presión sobre el heap y capacidad de reducir asignaciones innecesarias. Cuando corresponda, allocation, máxima residency, actividad del garbage collector y tiempo de ejecución complementan la comparación. Ninguna métrica aislada permite afirmar que una estrategia sea superior: menor allocation puede coexistir con mayor tiempo de CPU, y ejecutar antes puede reducir thunks a costa de producir valores que luego no se utilizan.

En consecuencia, el marco teórico conduce a una pregunta experimental concreta: cómo cambia la representación y el ciclo de vida del trabajo pendiente cuando se modifica la demanda de evaluación. Core y STG permiten observar las decisiones del compilador; los perfiles del heap muestran sus consecuencias espaciales; y el IR de bajo nivel permite estudiar cómo esas decisiones terminan expresándose sobre una máquina convencional. La estrategia híbrida de Maessen amplía el espacio de

diseño y sirve como antecedente para interpretar los resultados sin presuponer que lazy o strict deban ganar en todos los programas.

Cierre del marco teórico. Las seis unidades establecen una cadena causal que guía la PPS: estrategia de evaluación -> thunks y closures -> asignación y retención -> trabajo del RTS -> rendimiento observable. Los experimentos buscarán evidencia en cada nivel de esa cadena, manteniendo el alcance definido y explicitando las limitaciones de cada comparación.

Referencias

- Apfelmus, H. (2014). How does lazy evaluation work in Haskell? <https://apfelmus.nfshost.com/articles/lazy-eval-intro.html>
- Apfelmus, H. (2015). Non-strict semantics of Haskell. <https://apfelmus.nfshost.com/articles/non-strict-semantics.html>
- GHC Team. (s. f.-a). GHC User's Guide 9.10.3: Debugging the compiler. Consultado el 7 de septiembre de 2026. https://downloads.haskell.org/~ghc/9.10.3/docs/users_guide/debugging.html
- GHC Team. (s. f.-b). GHC User's Guide 9.10.3: Optimisation (code improvement). Consultado el 7 de septiembre de 2026. https://downloads.haskell.org/~ghc/9.10.3/docs/users_guide/using-optimisation.html
- GHC Team. (s. f.-c). GHC User's Guide 9.10.3: Runtime system (RTS) options. Consultado el 7 de septiembre de 2026. https://downloads.haskell.org/~ghc/9.10.3/docs/users_guide/runtime_control.html
- Maessen, J.-W. (2002). Hybrid eager and lazy evaluation for efficient compilation of Haskell [Tesis doctoral, Massachusetts Institute of Technology]. <https://dspace.mit.edu/entities/publication/ffc8c204-c55b-4783-947c-fdf298ee5228>
- Marlow, S. (Ed.). (2010). Haskell 2010 Language Report. <https://www.haskell.org/onlinereport/haskell2010/>
- Marlow, S., & Peyton Jones, S. (2004). Making a fast curry: Push/enter vs. eval/apply for higher-order languages. Proceedings of ICFP 2004, 4–15. <https://doi.org/10.1145/1016850.1016856>
- Marlow, S., & Peyton Jones, S. (2012). The Glasgow Haskell Compiler. En A. Brown & G. Wilson (Eds.), The Architecture of Open Source Applications, Vol. II. <https://aosabook.org/en/v2/ghc.html>
- Marlow, S., Rodriguez Yakushev, A., & Peyton Jones, S. (2007). Faster laziness using dynamic pointer tagging. Proceedings of ICFP 2007, 277–288. <https://doi.org/10.1145/1291151.1291194>
- Marlow, S., Harris, T., James, R. P., & Peyton Jones, S. (2008). Parallel generational-copying garbage collection with a block-structured heap. Proceedings of ISMM 2008, 11–20. <https://doi.org/10.1145/1375634.1375637>
- Mitchell, N. (2013). Chasing a space leak in Shake. <https://neilmitchell.blogspot.com/2013/02/chasing-space-leak-in-shake.html>
- Peyton Jones, S. L. (1987). The implementation of functional programming languages. Prentice Hall. <https://simon.peytonjones.org/slpj-book-1987/>
- Peyton Jones, S. L. (1992). Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine. Journal of Functional Programming, 2(2), 127–202. <https://doi.org/10.1017/S0956796800000319>
- Tofte, M., & Talpin, J.-P. (1997). Region-based memory management. Information and Computation, 132(2), 109–176. <https://doi.org/10.1006/inco.1996.2613>